MEMORIA
PROYECTO

DesignWorks

Luis Imaicela
PROYECTO FINAL DAM
24/03/2026

Santiago Roman Viguera
TUTOR DEL PROYECTO

LUIS IMAICELA
Diseño y Comunicación

INDICE

1. INTRODUCCIÓN
   1.1 Descripción general del proyecto 1
   1.2 Motivación y justificación 2
   1.3. Objetivos del proyecto 2
   1.4. Alcance 2

2. PLANIFICACIÓN DEL PROYECTO
   2.1. Cronograma de actividades 3
   2.2. Metodología de desarrollo 3

3. FASE DE ANÁLISIS
   3.1. Requisitos funcionales 4
   3.2. Requisitos no funcionales 8
   3.3. Identificación de roles 10
   3.4. Casos de uso 11

4. FASE DE DISEÑO
   4.1. Arquitectura del sistema 13
   4.2. Arquitectura interna del backend 14
   4.3. Diseño de base de datos 15
   4.4. Diagramas UML 17
   4.5. Endpoints REST 24
   4.6. Seguridad Jwt y matriz de permisos 25
   4.7. Diseño UI/UX 27

5. IMPLEMENTACIÓN
   5.1. Tecnologías utilizadas 28
   5.2. Funcionalidades implementadas 29
   5.3. Infraestructura y despliegue 29

6. RESULTADOS Y ESTADO DEL PROYECTO
   6.1. Progreso global 30
   6.2. Métricas de código 30
   6.3. Decisiones técnicas relevantes 30

7. CONCLUSIONES
   7.1. Valoración del proyecto 31
   7.2. Lecciones aprendidas 31
   7.3. Trabajo futuro 31

8. BIBLIOGRAFÍA Y WEBGRAFÍA
   8.1. Documentación oficial 32
   8.2. Libros y artículos de referencia 32
   8.3. Recursos complementarios 32

Proyecto final DAM

1. INTRODUCCIÓN

1.1 DESCRIPCIÓN GENERAL DEL PROYECTO

DesignWorks es una aplicación móvil desarrollada como Proyecto Final del Ciclo Formativo de Desarrollo de Aplicaciones Multiplataforma (DAM) del curso 2025/2026. Su objetivo es digitalizar y centralizar la gestión de los encargos y proyectos de diseño en un estudio de diseño gráfico, facilitando la organización y la comunicación entre los distintos miembros del equipo en general.

La aplicación permite a los administradores del estudio crear y supervisar trabajos, asignar diseñadores, definir requisitos y hacer seguimiento del estado de cada proyecto. Los diseñadores pueden consultar sus tareas asignadas, añadir comentarios y actualizar el estado de sus trabajos conforme al flujo de trabajo establecido.

1.2 MOTIVACIÓN Y JUSTIFICACIÓN

La idea surge de la necesidad real y propia de los estudios de diseño de contar con una herramienta propia que centralice la información de cada encargo, evitando el uso de hojas de cálculo, correos electrónicos o aplicaciones genéricas que no se adaptan al flujo de trabajo específico del sector.

Desde la perspectiva formativa, el proyecto permite integrar y aplicar los conocimientos adquiridos a lo largo del ciclo en áreas como el desarrollo backend con Spring Boot, el desarrollo móvil con Flutter, el diseño de bases de datos relacionales, la implementación de sistemas de autenticación y seguridad, y el diseño UI/UX con Figma.

1.3. OBJETIVOS DEL PROYECTO

- Desarrollar una API REST segura con Spring Boot que gestione usuarios, trabajos, comentarios, requisitos e historial de estados.
- Implementar un sistema de autenticación y autorización basado en JWT con control de acceso por roles (Administrador y Diseñador).
- Crear una aplicación móvil en Flutter que consuma la API y ofrezca una interfaz intuitiva adaptada al flujo de trabajo de un estudio de diseño.
- Aplicar un diseño UI/UX profesional usando Material Design 3, elaborado previamente en Figma.
- Garantizar la trazabilidad de los trabajos mediante un historial completo de cambios de estado.
- Desplegar la infraestructura de forma portable utilizando Docker Compose.

1.4. ALCANCE

El sistema cubre los siguientes módulos funcionales:

- Autenticación segura con JWT y gestión de sesión en el cliente móvil.
- Gestión completa de trabajos: creación, listado, detalle y cambio de estado.
- Sistema de participantes por trabajo con control de permisos.
- Gestión de requisitos y comentarios asociados a cada trabajo.
- Historial de estados para trazabilidad y auditoría.
- Interfaz diferenciada por rol: Administrador y Diseñador.

Quedan fuera del alcance actual: la gestión de archivos adjuntos, el módulo de notificaciones push, la edición de perfil de usuario desde la app y el soporte iOS en producción.

2. PLANIFICACIÓN DEL PROYECTO

2.1 CRONOGRAMA DE ACTIVIDADES

El proyecto se organizó según el siguiente cronograma, establecido desde la fase de análisis:

Nombre de la tarea	Plazo
Defensa	7 de abril de 2026 → 16 de abril de 2026
Validación	1 de abril de 2026 → 3 de abril de 2026
Entrega final	27 de marzo de 2026
Desarrollo avanzado	3 de marzo de 2026 → 30 de marzo de 2026
Entrega 2	2 de marzo de 2026
Desarrollo inicial **(Backend + Frontend básico)**	2 de febrero de 2026 → 27 de febrero de 2026
Entrega 1	30 de enero de 2026
Fase de diseño	16 de diciembre de 2025 → 29 de enero de 2026
Fase de análisis	30 de octubre de 2025 → 15 de diciembre de 2025
Aceptación del proyecto	25 de octubre de 2025 → 29 de octubre de 2025
Propuesta inicial	1 de octubre de 2025 → 24 de octubre de 2025

2.2 METODOLOGÍA DE DESARROLLO

Se ha seguido una metodología iterativa e incremental, partiendo de las fases de análisis y diseño y avanzando progresivamente hacia la implementación. Cada iteración ha sido revisada con el tutor del proyecto, incorporando correcciones y mejoras. El control de versiones se ha gestionado con Git, usando la rama main como rama de entrega y ramas feature/ para el desarrollo de funcionalidades concretas.

3. FASE DE ANÁLISIS

La app **DesignWorks** permite gestionar los trabajos y encargos de un estudio de diseño gráfico. Su objetivo es centralizar la organización de proyectos y la comunicación entre los miembros del equipo. Los requisitos funcionales se dividen por módulos.

3.1 REQUISITOS FUNCIONALES

3.1.1 Autenticación y gestión de usuarios

RF1. Registro / alta de usuarios (solo backend / admin).

- Por razones de tiempo y simplicidad, la creación de usuarios se realizará exclusivamente desde el **backend** (fuera de la app móvil).
- El sistema debe permitir dar de alta usuarios con: nombre, email, rol (Administrador/Diseñador), contraseña.
- La contraseña se almacenará en la BD cifrada (hash).
- Solo un usuario con rol **Administrador** (en el backend) podrá **crear, modificar o desactivar** usuarios.
- En la aplicación móvil, los usuarios únicamente podrán **iniciar sesión** con las credenciales que se les haya asignado previamente.

RF2. Inicio de sesión con JWT

- La app debe ofrecer una pantalla de **login** en la que el usuario introduce email y contraseña.
- El backend valida las credenciales y, si son correctas, devuelve un **token JWT**.
- La app almacena el token de forma segura (flutter_secure_storage) y lo enviará en cada petición.
- Si el token es inválido o caducado, la app mostrará un mensaje y devolverá al login.

RF3. Gestión básica de perfil

- El usuario podrá consultar sus datos básicos (nombre, email, rol) desde la app.
- El usuario podrá cambiar su contraseña (opcional como mejora futura pero me ha parecido interesante mencionarlo).

3.1.2 Gestión de trabajos

RF4. Creación de trabajo (Administrador)

- El Administrador puede crear un nuevo trabajo indicando como mínimo:
  - Título del trabajo.
  - Cliente.
  - Descripción breve / briefing.
  - Prioridad (baja, media, alta, urgente).
  - Fecha de creación (automática).
  - Fecha límite (opcional).

- Al crear el trabajo, su **estado inicial** será “Creado”.

RF5. Edición y eliminación de trabajos (Administrador)

- El Administrador puede editar los datos de un trabajo mientras no esté en estado “Entregado” o “Cancelado”.
- El Administrador puede marcar un trabajo como “Cancelado” si finalmente no se realiza (en lugar de borrarlo físicamente).

RF6. Asignación de participantes a un trabajo

- El Administrador podrá seleccionar uno o varios diseñadores y asignarlos a un trabajo.
- Se registrará el rol del diseñador en ese trabajo (por ejemplo, diseñador principal, apoyo, ilustrador, maquetador…).
- Los diseñadores solo verán en “Mis trabajos” aquellos en los que sean participantes

RF7. Definición de requisitos / briefs del trabajo

- Para cada trabajo, se podrán definir uno o varios **requisitos** (briefs parciales o puntos clave):
  - Descripción del requisito.
  - Posible adjunto (URL a referencia, archivo, etc.).

- Los requisitos serán visibles para todos los participantes asignados participantes.

3.1.3 Flujo de estados y seguimiento

RF8. Gestión de estados del trabajo

Estados principales:

- Creado
- En progreso
- En revisión
- Entregado
- Cancelado

Reglas mínimas del flujo:

- De “Creado” → se puede pasar a “En progreso” o “Cancelado”.
- De “En progreso” → se puede pasar a “En revisión” o “Cancelado”.
- De “En revisión” → se puede pasar a “En progreso” (si hay cambios) o “Entregado”.
- “Entregado” y “Cancelado” se consideran estados finales (no se puede cambiar más, esto lo he establecido así por ahora, en un futuro puede cambiarse).

RF9. Cambio de estado (Administrador y Diseñador)

- Un **Diseñador participante** puede cambiar el estado del trabajo si tiene permisos (por ejemplo, de “En progreso” a “En revisión”).

- El **Administrador** puede forzar cambios de estado en cualquier momento.

- Cada cambio de estado debe registrar en la tabla de historial:
  - Trabajo.
  - Estado nuevo.
  - Fecha y hora.
  - Usuario que realiza el cambio.
  - Motivo o comentario de cambio (opcional pero muy útil en un estudio real).

3.1.4 Comentarios, revisiones y comunicación interna

RF10. Comentarios / revisiones sobre el trabajo

- Los participantes (Administrador y Diseñadores asignados) pueden añadir comentarios al trabajo.
- Cada comentario tiene: texto, usuario, fecha/hora.
- Los comentarios se listan cronológicamente, a modo de “timeline” del trabajo.

3.1.5 Consulta y filtrado de trabajos

RF11. Vista general de trabajos (Administrador)

- Lista de todos los trabajos registrados.
- Posibilidad de filtrar por: estado, prioridad, cliente, rango de fechas.
- Posibilidad de ordenar por fecha de creación, fecha límite, prioridad, etc.

RF12. “Mis trabajos” (Diseñador)

- Cada Diseñador verá una lista de los trabajos en los que está asignado.
- Podrá filtrar por estado (por ejemplo, ver solo “En progreso”).

RF13. Detalle de trabajo

- Al entrar al detalle se verá:
  - Datos básicos: título, cliente, prioridad, descripción.
  - Requisitos / briefs.
  - Participantes.
  - Estado actual.
  - Historial de estados.
  - Comentarios.

3.1.6 Historial y auditoría

RF14. Consulta de historial de estados

- Desde el detalle del trabajo, se podrá ver el **historial de estados** con fecha y usuario.
- También habrá un endpoint tipo “GET /historial/{trabajoId}” para recuperar esta información.

3.2 REQUISITOS NO FUNCIONALES

3.2.1. Usabilidad y experiencia de usuario

- **RNF1. Interfaz clara y consistente**: diseño basado en **Material Design 3** con modo claro/oscuro, pensada para uso intensivo en un estudio.

- **RNF2. Navegación simple**:
  - Menú principal: “Mis trabajos”, “Todos los trabajos” (solo admin), “Perfil”·.
  - Flujo directo: lista → detalle → cambio de estado / comentario.

- **RNF3. Validación de formularios**: campos obligatorios señalados, mensajes de error claros (ej. “Título obligatorio”, “Formato de email inválido”).

3.2.2. Rendimiento

- **RNF4. Tiempo de respuesta**: las operaciones comunes (consultar trabajos, cambiar estado, añadir comentario) deberían responder en **< 300 ms** en condiciones normales de red local o WiFi estándar.

- **RNF5. Eficiencia en consultas**: uso de índices adecuados en la BD (por ejemplo en campos de búsqueda frecuentes: estado, prioridad, fecha, cliente).

3.2.3. Seguridad

- **RNF6. Autenticación y autorización**:
  - Acceso a la API mediante **JWT**.
  - Protección de endpoints según rol (Admin vs Diseñador) mediante Spring Security.

- **RNF7. Protección de credenciales**:
  - Contraseñas cifradas con algoritmo robusto (BCrypt).
  - Token almacenado en “flutter_secure_storage” en el móvil.

- **RNF8. Control de permisos**: un usuario solo puede ver y modificar lo que le corresponde (por ejemplo, un diseñador no ve trabajos de otros si no está asignado).

3.2.4. Confiabilidad y mantenimiento

- **RNF9. Integridad de datos**:
  - Uso de transacciones en operaciones críticas (creación de trabajo + asignación de participantes, cambios de estado…).

- **RNF10. Registro de errores (logging)**:
  - La API debe registrar errores y excepciones para facilitar el diagnóstico.

- **RNF11. Mantenibilidad**:
  - Arquitectura por capas (controlador, servicio, repositorio).
  - Código documentado y versionado en GitHub.

3.2.5. Portabilidad y despliegue

- **RNF12. Portabilidad de backend**:
  - Backend empaquetado con **Docker Compose** (contenedor API + BD), lo que permite ejecutarlo en diferentes máquinas de forma homogénea.

- **RNF13. Portabilidad de frontend**:
  - App desarrollada en **Flutter** preparada para Android, con posibilidad futura de iOS sin reescribir toda la lógica.

3.3 IDENTIFICACIÓN DE ROLES

3.3.1 Actor: Administrador

Descripción

Persona responsable de la organización de los trabajos dentro del estudio de diseño (por ejemplo, el jefe de estudio o project manager).

Objetivos

- Tener una visión global de todos los trabajos.
- Asignar tareas a los diseñadores.
- Supervisar el avance y garantizar que se cumplan los plazos.

Permisos principales

- Crear, editar y cancelar trabajos.
- Asignar y modificar participantes.
- Definir o editar requisitos del trabajo.
- Cambiar estados de cualquier trabajo.
- Consultar todos los historiales y comentarios.
- Dar de alta o desactivar usuarios (opcional según alcance final).

3.3.2 Actor: Diseñador

Descripción

Miembro del equipo creativo del estudio que realiza los trabajos asignados.

Objetivos

- Ver claramente qué trabajos tiene asignados, con prioridades y plazos.
- Comunicar avances y dudas mediante comentarios.
- Indicar cambios de estado acorde al flujo real de trabajo.

Permisos principales

- Ver la lista de “Mis trabajos” y el detalle de cada uno.
- Ver requisitos, participantes y comentarios de sus trabajos.
- Añadir comentarios / revisiones.
- Cambiar estado de sus trabajos dentro del flujo permitido (por ejemplo, de “En progreso” a “En revisión”).
- Consultar el historial de estados de sus trabajos.
  (Importante: el diseñador NO puede crear trabajos nuevos)

3.4 CASOS DE USO

3.4.1 Lista de casos de uso principales

ID	Caso de uso	Actor principal
CU1	Iniciar sesión	Administrador / Diseñador
CU2	Gestionar usuarios (backend)	Administrador
CU3	Crear trabajo	Administrador
CU4	Editar / cancelar trabajo	Administrador
CU5	Asignar participantes a un trabajo	Administrador
CU6	Definir requisitos del trabajo	Administrador
CU7	Consultar todos los trabajos	Administrador
CU8	Consultar mis trabajos	Diseñador
CU9	Ver detalle de trabajo	Administrador / Diseñador
CU10	Cambiar estado de trabajo	Administrador / Diseñador
CU11	Añadir comentario / revisión	Administrador / Diseñador
CU12	Consultar historial de estados	Administrador / Diseñador

3.4.2. Especificaciones textuales de casos de uso

CU3 — Crear trabajo
Actor principal: Administrador
Objetivo: Registrar un nuevo trabajo en el sistema para su posterior asignación y seguimiento.
Precondiciones: El Administrador ha iniciado sesión correctamente.
Flujo principal: 1. El Administrador accede a la opción 'Nuevo trabajo'. 2. El sistema muestra el formulario de creación. 3. El Administrador introduce título, cliente, descripción, prioridad y fecha límite (opcional). 4. El Administrador confirma la creación. 5. El sistema valida los datos (campos obligatorios, formato de fecha). 6. El sistema guarda el trabajo en la BD con estado inicial CREADO y fecha de creación actual. 7. El sistema muestra confirmación y el nuevo trabajo aparece en la lista.
Postcondiciones: El trabajo queda registrado y disponible para su asignación y gestión posterior.
Flujo alternativo: Si hay errores de validación, el sistema informa de los campos incorrectos y vuelve al formulario sin perder la información introducida.

CU8 — Consultar mis trabajos
Actor principal: Diseñador
Precondición: El Diseñador ha iniciado sesión correctamente.
Flujo principal: 1. El Diseñador selecciona la opción 'Mis trabajos'. 2. El sistema consulta en la BD todos los trabajos en los que el usuario está asignado. 3. El sistema muestra la lista filtrable por estado y prioridad.
Postcondición: El Diseñador visualiza únicamente los trabajos en los que participa.

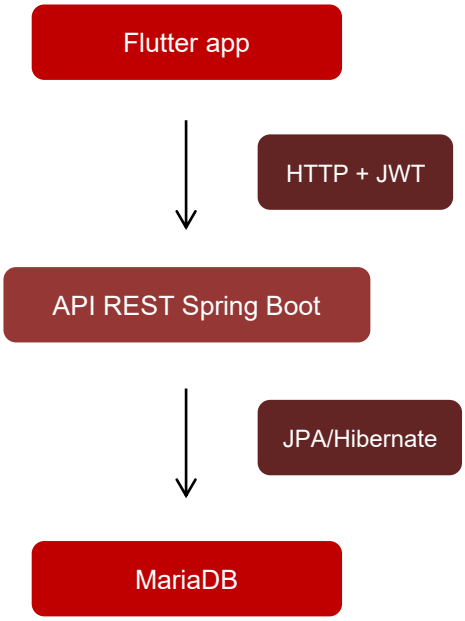
CU10 — Cambiar estado de trabajo
Actor principal: Diseñador (o Administrador)
Precondiciones: El usuario ha iniciado sesión. El usuario es participante del trabajo (o es Administrador).
Flujo principal: 1. El usuario abre el detalle de un trabajo. 2. El usuario selecciona 'Cambiar estado'. 3. El sistema muestra los estados disponibles desde el estado actual. 4. El usuario elige el nuevo estado e introduce opcionalmente un motivo. 5. El sistema valida que la transición es permitida según las reglas de negocio. 6. El sistema actualiza el estado y registra la entrada en historial_estados. 7. El sistema muestra confirmación.
Flujo alternativo: Si la transición no está permitida, el sistema muestra un mensaje de error y no realiza cambios.

4. FASE DE DISEÑO

Siguiendo el cronograma establecido en la fase de análisis, la fase de diseño organiza los siguientes bloques: arquitectura del sistema, diseño de BD (modelo E-R + relacional), diagramas UML, endpoints REST, diseño UI/UX y matriz de permisos con seguridad JWT.

4.1 ARQUITECTURA DEL SISTEMA

En mi caso el sistema de la app lo he establecido en tres capas diferentes, esto lo hago pensando en conseguir una arquitectura modular y escalable.



- **PRIMERA CAPA - Aplicación móvil (Frontend – Flutter)**

  Es la responsable e la experiencia de usuario y de todas las interacciones directas con el sistema. Gestiona autenticación mediante JWT, consulta y manipulación de trabajos, comentarios y estados.

  Funciones principales:
  - Autenticación (JWT).
  - Visualización de trabajos (“Mis trabajos” y “Todos los trabajos”).
  - Detalle del trabajo.
  - Cambio de estado.
  - Gestión de comentarios.
  - Consulta del historial.

- **SEGUNDA CAPA - API REST implementada en Spring Boot**

  Actúa como núcleo del sistema. Procesa la autenticación, aplica reglas de negocio, controla los roles y ejecuta toda la lógica relacionada con trabajos, requisitos y participantes.

  Se encarga de:
  - Autenticación y generación de tokens.
  - Gestión de trabajos, requisitos y participantes.
  - Persistencia de datos.
  - Validación del flujo de estados.
  - Control de roles (Administrador / Diseñador).

- **TERCERA CAPA - Base de datos MariaDB**

  Es donde planteo almacenar de forma estructurada toda la información del sistema, garantizando integridad, relación entre entidades y trazabilidad mediante el historial de estados.

  Contiene información estructurada de:
  - Usuarios
  - Trabajos
  - Participantes
  - Comentarios
  - Requisitos
  - Historial de estados

4.2. ARQUITECTURA INTERNA DEL BACKEND

El backend se organiza en una estructura por capas que favorece la mantenibilidad:

Paquete	Responsabilidad
config	Parámetros globales del sistema (CORS, beans de configuración).
security	Manejo de JWT, filtros de autenticación (JwtAuthFilter) y reglas de Spring Security.
controllers	Puntos de entrada REST. Reciben peticiones, coordinan acciones y devuelven respuestas.
services	Lógica de negocio centralizada: validaciones, reglas de transición de estado, permisos.
repositories	Acceso a la base de datos mediante interfaces Spring Data JPA.
entities	Modelos persistentes del dominio mapeados como entidades JPA.
dto	Objetos de transferencia de datos optimizados para peticiones y respuestas REST.
exceptions	Manejo estructurado de errores: excepciones personalizadas y @ControllerAdvice global.

```
designworks
│
│  ── config
│  ── security (JWT, authentication filters)
│  ── controllers (REST Controllers)
│  ── services (Business logic)
│  ── repositories (JPA Repositories)
│  ── entities (JPA entity models)
│  ── dto (Data Transfer Objects)
│  ── exceptions
```

4.3. DISEÑO DE BASE DE DATOS

A partir de los requisitos funcionales que establecí en la fase de análisis, se han identificado las entidades principales, sus relaciones y la posterior transformación al modelo relacional. Esto permite asegurar integridad referencial y una estructura preparada para crecer sin perder orden.

4.3.1. Entidades principales

Las entidades definen los elementos clave de la app, entre ellos se encuentran:

- **Usuario:** representa a los actores del sistema (administradores y diseñadores). Incluye información de identificación, credenciales (almacenadas como "contraseña_hash") y el estado de actividad.
- **Trabajo:** núcleo del sistema, agrupa toda la información de un encargo de diseño (cliente, prioridad, fechas, estado actual, descripción y el usuario que lo creó).
- **Participantes:** modela la relación entre trabajos y usuarios, indicando qué usuarios participan en cada trabajo y su "rol_en_trabajo".
- **Comentario:** permite registrar las aportaciones, revisiones y comunicación entre participantes sobre un trabajo.
- **Requisito:** recoge las necesidades específicas del trabajo, con posibilidad de asociar adjuntos (adjunto_url).
- **Historial de Estados:** almacena todos los cambios de estado de un trabajo, quién los realizó y el motivo, proporcionando una trazabilidad completa del ciclo de vida.

Estas entidades cubren tanto la gestión operativa (usuarios, trabajos, participantes) como el seguimiento detallado (comentarios, requisitos, historial).

4.3.2. Relaciones del modelo E-R

En el modelo Entidad–Relación se representan las asociaciones entre las entidades definidas, es aquí donde se reflejan todos los conocimientos adquiridos en la asignatura de Base de Datos:

- Un Usuario puede crear muchos "Trabajos" → "relación 1:N" entre "Usuario" y "Trabajo" (mediante el campo "trabajo_creado_por").
- La participación en trabajos se representa como una "relación N:M" entre "Usuario y Trabajo", resuelta mediante la entidad intermedia "Trabajo_Participantes" (trabajo_id, usuario_id, rol_en_trabajo).
- Cada Trabajo se relaciona con múltiples Requisitos, Comentarios y entradas en el Historial de Estados → "relaciones 1:N" desde Trabajo hacia estas entidades.
- Cada registro del Historial de Estados está asociado al Usuario que realiza el cambio, reforzando la responsabilidad y trazabilidad (FK usuario_id).

4.3.3. Modelo relacional — Tablas

Tabla	Campos principales	Tipos clave
USUARIOS	id, nombre, email (UNIQUE), rol, contraseña_hash, activo	INT PK, VARCHAR(150), VARCHAR(20), VARCHAR(255)
TRABAJOS	id, titulo, cliente, prioridad, fecha_inicio, fecha_fin, estado_actual, descripcion, creado_por (FK)	ENUM prioridad, ENUM estado, DATETIME, INT FK
TRABAJO_PARTICIPANTES	trabajo_id (FK), usuario_id (FK), rol_en_trabajo — PK compuesta	INT FK×2, VARCHAR(50)
COMENTARIOS	id, trabajo_id (FK), usuario_id (FK), fecha, texto	INT PK, INT FK×2, DATETIME, TEXT
REQUISITOS	id, trabajo_id (FK), descripcion, adjunto_url	INT PK, INT FK, TEXT, VARCHAR(255)
HISTORIAL_ESTADOS	id, trabajo_id (FK), estado (ENUM), fecha, usuario_id (FK), motivo	INT PK, INT FK×2, ENUM, DATETIME, TEXT

4.4. DIAGRAMAS UML

Se plantea la estructura principal del sistema de **DesignWorks** en términos de clases de dominio, servicio y controladores, así como las relaciones entre estos componentes. Ha sido una tarea que me ha costado bastante tiempo debido a que acotar todo para que funcione correctamente y tenga sentido es un poco tedioso y en principio difícil de entender.

4.4.1. Diagrama de clases

- **Clases de dominio**

  Las clases de dominio representan los elementos principales del modelo de negocio. Contienen los atributos y comportamientos propios de las entidades que forman parte del sistema, proporcionando una estructura lógica para los datos que se gestionan.

  - **Usuario**: Representa a los usuarios del sistema (administradores y diseñadores). Contiene los datos básicos de identificación y autenticación (nombre, email, rol, contraseña cifrada, estado activo).
  - **Trabajo:** Es la entidad central del sistema. Modela los encargos de diseño con información como título, cliente, prioridad, fechas, estado actual, descripción y el usuario que crea el trabajo (administrador de ese trabajo).
  - **Requisito:** Describe las necesidades específicas asociadas a un trabajo (por ejemplo, entregables, formatos o indicaciones del cliente), pudiendo incluir ficheros adjuntos mediante una URL.
  - **Comentario:** Registra las revisiones y la comunicación entre los participantes de un trabajo. Cada comentario está vinculado a un trabajo y a un usuario, e incluye la fecha y el texto del mensaje.
  - **Participante**: Modela la relación entre usuarios y trabajos. Permite indicar qué usuarios participan en cada trabajo y con qué rol (por ejemplo, diseñador asignado), resolviendo así la relación N..M entre Usuario y Trabajo.
  - **HistorialEstado:** Guarda todos los cambios de estado que sufre un trabajo a lo largo de su ciclo de vida. Cada registro indica el nuevo estado, la fecha del cambio, el usuario que lo realiza y el motivo.

- **Clases de servicio**

  Estas clases encapsulan la lógica de negocio y las operaciones que se realizan sobre las entidades del dominio. Actúan como intermediarias entre los controladores y la capa de acceso a datos, garantizando reglas, validaciones y procesos internos que deben cumplirse antes de modificar o consultar la información.

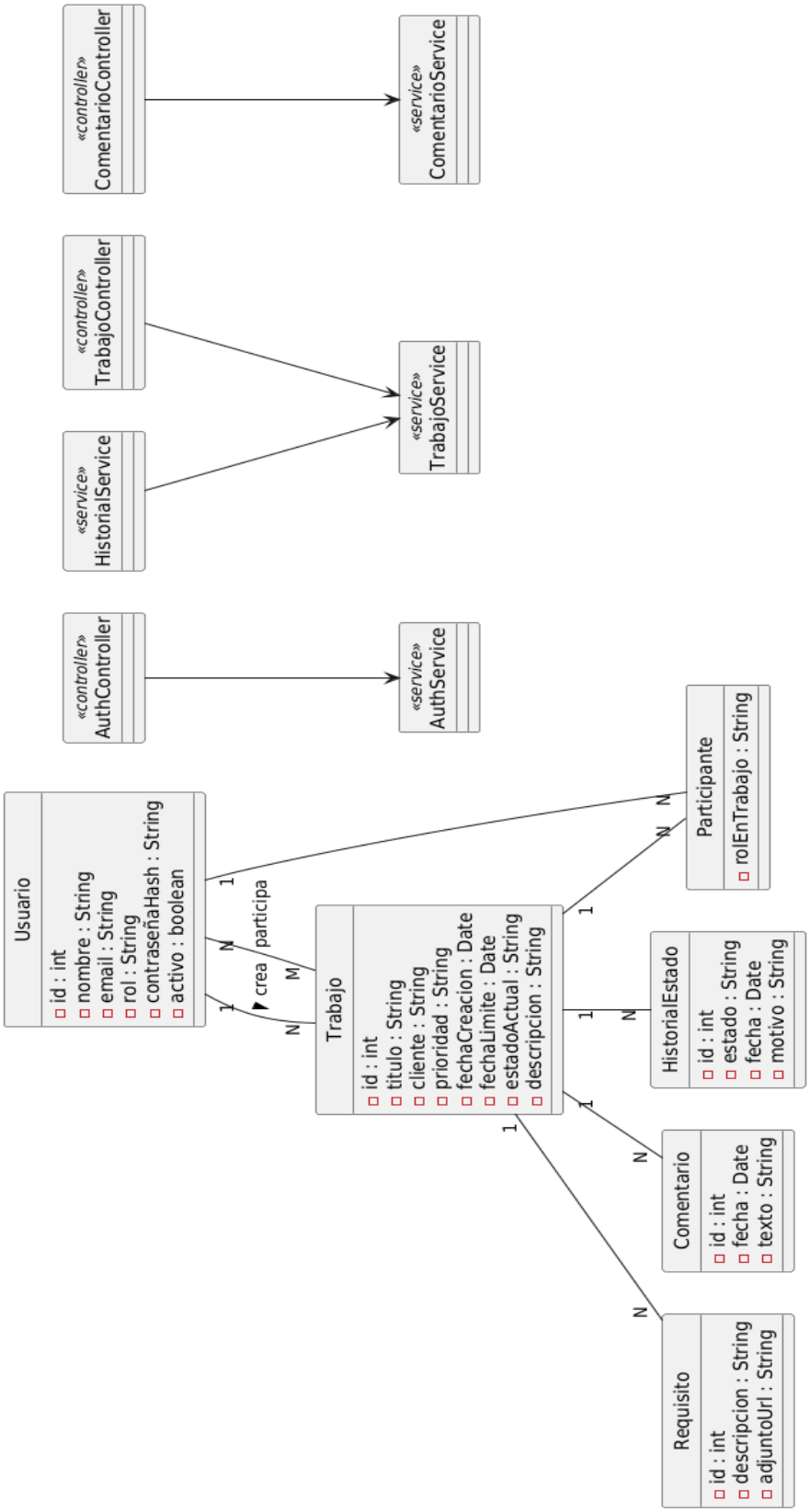
  - **AuthService**: Encapsula la lógica de autenticación y autorización: validación de credenciales, generación y verificación de tokens JWT y gestión de roles.
  - **TrabajoService:** Contiene la lógica de negocio relacionada con los trabajos: creación, edición, cancelación, cambio de estado, asignación de participantes y validación de reglas (por ejemplo, un único administrador por trabajo).
  - **ComentarioService:** Gestiona la creación y consulta de comentarios asociados a los trabajos, asegurando que el usuario tenga permisos sobre el trabajo.
  - **HistorialService:** Se encarga de registrar y consultar el historial de estados de un trabajo, garantizando la trazabilidad de todos los cambios realizados.

- **Clases de controladores**

  Gestionan la comunicación con el exterior. Reciben peticiones del cliente (API o interfaz gráfica), coordinan las acciones necesarias y delegan la lógica en los servicios correspondientes. Además, son las responsables de preparar y devolver las respuestas adecuadas al usuario o al sistema que realiza la solicitud.

  - **AuthController**: Expone los endpoints REST relacionados con el inicio de sesión y la obtención de tokens (por ejemplo, /auth/login).
  - **TrabajoController:** Ofrece los endpoints para gestionar trabajos y estados (crear, editar, listar, cambiar estado, asignar participantes, etc.).
  - **ComentarioController:** Proporciona los endpoints para añadir y consultar comentarios vinculados a un trabajo concreto.

- **Relaciones principales del diagrama de clases**
  - Un Trabajo se relaciona con Requisito, Comentario y HistorialEstado mediante asociaciones 1..N, ya que un trabajo puede tener múltiples requisitos, comentarios y registros de historial.
  - La relación entre Usuario y Trabajo es N..M a través de la clase Participante, que actúa como entidad intermedia y añade el rol del usuario en el trabajo.
  - Las clases de servicio se relacionan con las clases de dominio para aplicar la lógica de negocio, y las clases de controlador utilizan los servicios para atender las peticiones de la aplicación móvil.


«controller»
ComentarioController
«service»
ComentarioService
«controller»
TrabajoController
«service»
HistorialService
«service»
TrabajoService
«controller»
AuthController
«service»
AuthService
Usuario
id : int
nombre : String
email : String
rol : String
contraseñaHash : String
activo : boolean
crea
participa
Trabajo
id : int
titulo : String
cliente : String
prioridad : String
fechaCreacion : Date
fechaLimite : Date
estadoActual : String
descripcion : String
Participante
rolEnTrabajo : String
HistorialEstado
id : int
estado : String
fecha : Date
motivo : String
Comentario
id : int
fecha : Date
texto : String
Requisito
id : int
descripcion : String
adjuntoUrl : String

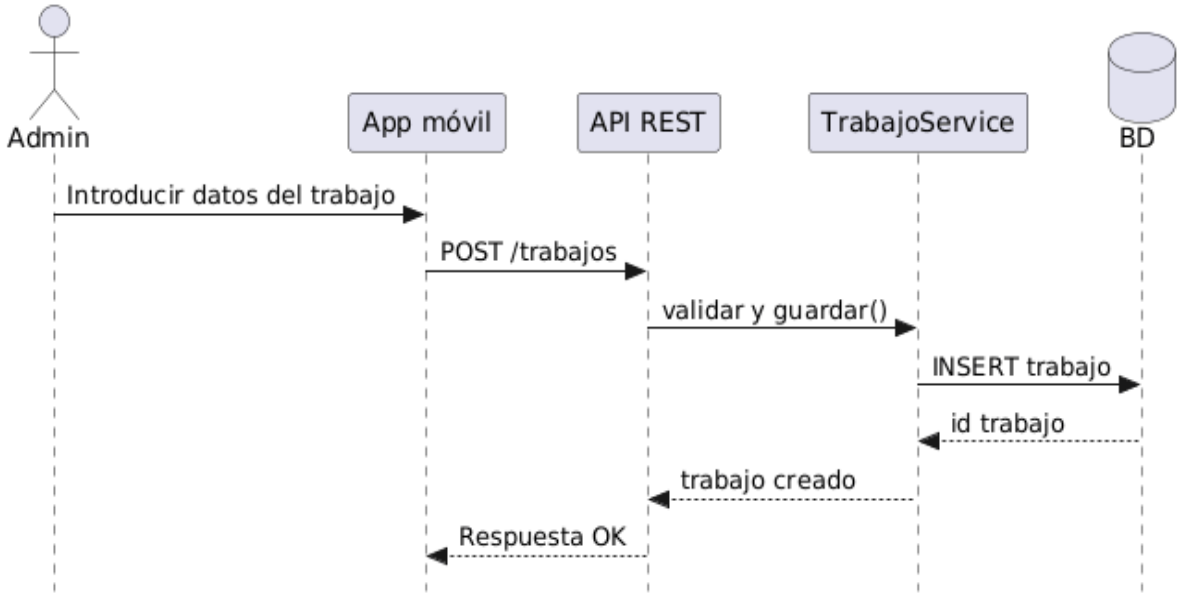
4.4.2. Diagramas de secuencia

Los diagramas de secuencia muestran el flujo de mensajes entre la App, la API y las capas internas del backend para cada caso de uso relevante. A continuación, he reflejado los diferentes flujos de mensajes existentes en la app.

- **CU3 – Crear trabajo**

  En este caso de uso, un administrador crea un nuevo trabajo desde la aplicación:

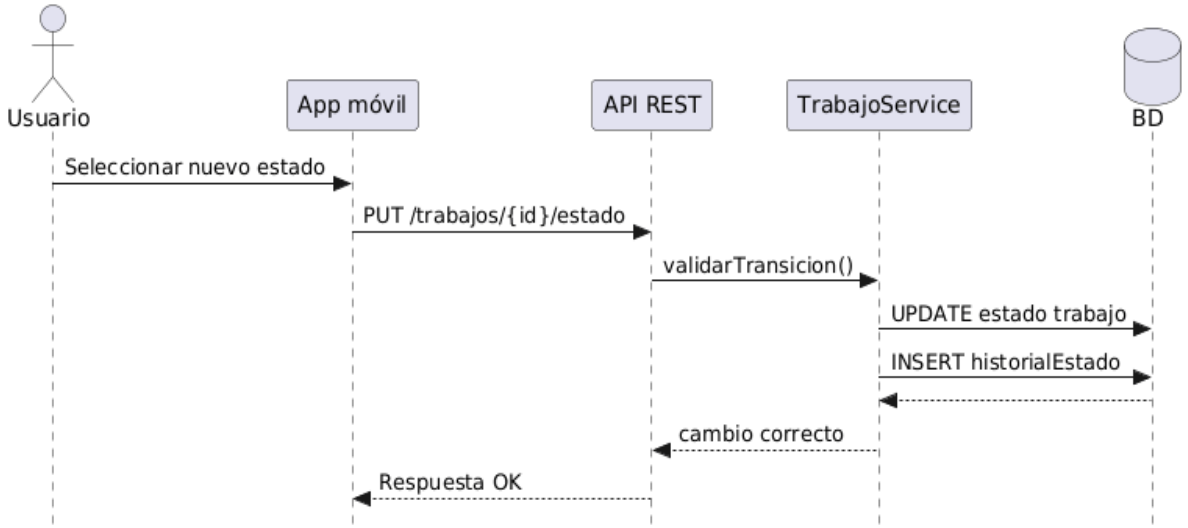
  - El **Administrador** introduce los datos del trabajo en la App móvil y envía el formulario.
  - La **App** realiza una petición POST /trabajos a la **API REST**, incluyendo el token JWT.
  - La API delega en "TrabajoService" la validación de los datos y de los permisos del usuario.
  - "TrabajoService" persiste el nuevo trabajo en la base de datos (mediante el repositorio/JPA).
  - La **BD** devuelve el identificador generado; el servicio responde a la **API**, y esta a su vez devuelve a la **App** la confirmación de creación del trabajo.



- **CU10 – Cambiar estado de trabajo**

  Este caso de uso describe la transición de estado de un trabajo:

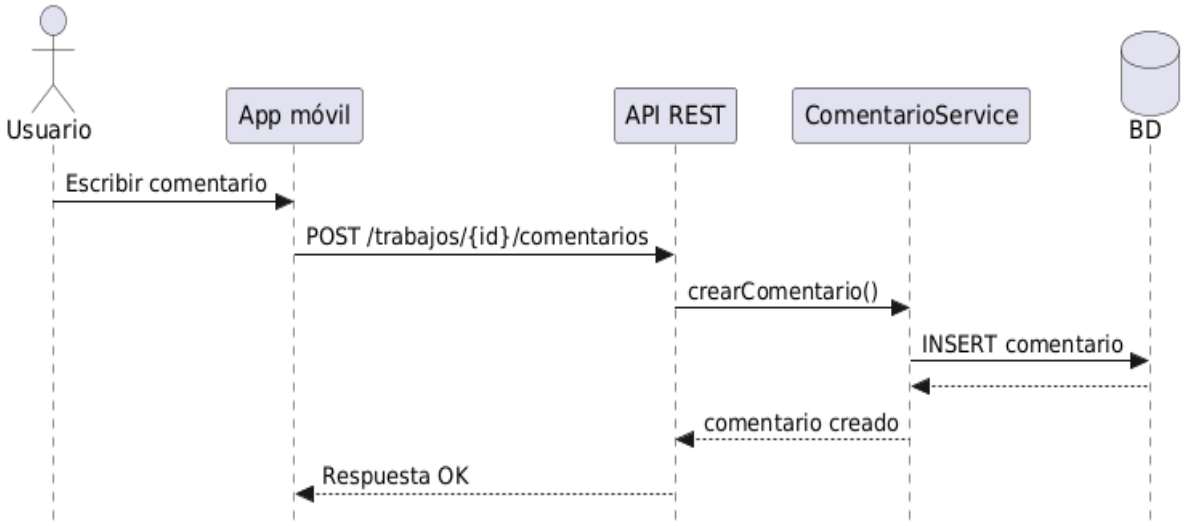
  1. El **Usuario** (normalmente el administrador del trabajo o un rol autorizado) selecciona un nuevo estado en la App.
  2. La **App** envía una petición PUT /trabajos/{id}/estado a la **API REST** con el nuevo estado.
  3. La **API** llama a “TrabajoService” para validar la transición de estado según las reglas definidas (por ejemplo, de En progreso solo puede pasar a “En revisión” o “Cancelado”).
  4. Si la transición es válida, “TrabajoService” actualiza el estado del trabajo en la BD.
  5. A continuación, el servicio registra el cambio en la tabla de “HistorialEstado” (insertando un nuevo registro con estado, fecha, usuario y motivo).
  6. Finalmente, la **BD** confirma las operaciones, el servicio responde a la **API** y esta devuelve el resultado a la App.



- **CU11 – Añadir comentario**

  Este caso de uso refleja la creación de un comentario sobre un trabajo:

  1. El **Usuario** escribe un comentario desde la **App**.
  2. La **App** envía una petición “POST /trabajos/{id}/comentarios” a la **API REST** con el contenido del comentario.
  3. La **API** deriva la operación a “ComentarioService”, que valida que el usuario pueda comentar ese trabajo.
  4. “ComentarioService” inserta el comentario en la **BD**, asociado al trabajo y al usuario.
  5. La **BD** confirma la inserción y el servicio devuelve la información a la **API**, que responde a la App con el comentario creado.

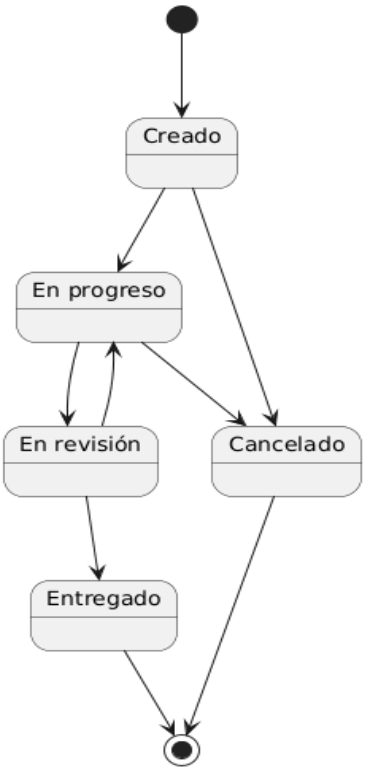


4.4.3. Diagrama de estados del trabajo

El diagrama de estados describe el ciclo de vida de un trabajo dentro del sistema **DesignWorks**.

- **Estados del trabajo**
  - **Creado:** estado inicial cuando se da de alta el trabajo, pero aún no se ha empezado a realizar.
  - **En progreso**: el trabajo está siendo ejecutado activamente por los diseñadores asignados.
  - **En revisión:** el trabajo está pendiente de revisión por parte del cliente o del administrador antes de considerarse finalizado.
  - **Entregado:** el trabajo ha sido finalizado y entregado; no se esperan más modificaciones (estado final "positivo").
  - **Cancelado:** el trabajo se ha detenido y no continuará (estado final "negativo").

- **Transiciones permitidas**
  - **Creado → En progreso / Cancelado**
    Desde un trabajo recién creado se puede comenzar la ejecución o cancelarlo directamente.
  - **En progreso → En revisión / Cancelado**
    Un trabajo en marcha puede pasar a revisión cuando se completa una versión, o puede cancelarse.
  - **En revisión → En progreso / Entregado**
    Tras revisar el trabajo, puede requerirse volver a "En progreso" para aplicar cambios, o bien marcarlo como "Entregado" si está aprobado.



4.5. ENDPOINTS REST

La API REST de **DesignWorks** sigue un diseño coherente orientado a recursos y se documenta mediante Swagger/OpenAPI. La autenticación se realiza mediante JWT; el token debe enviarse en la cabecera Authorization: Bearer {token} en todas las peticiones posteriores al login.

Método	Endpoint	Descripción	Rol requerido
POST	/auth/login	Autenticación. Request: {email, password}. Response: {token, rol}.	Público
GET	/auth/me	Datos del usuario autenticado.	Token válido
POST	/trabajos	Crear nuevo trabajo. El usuario creador se convierte en administrador del trabajo.	ADMIN
GET	/trabajos	Listar todos los trabajos con filtros opcionales por estado, prioridad, cliente y fechas.	ADMIN
GET	/trabajos/mis	Listar los trabajos asignados al usuario autenticado.	ADMIN / DISENADOR
GET	/trabajos/{id}	Detalle completo de un trabajo: info general, requisitos, comentarios, participantes y estado.	ADMIN o participante
PUT	/trabajos/{id}	Editar datos de un trabajo (restringido al administrador del trabajo).	ADMIN
PUT	/trabajos/{id}/estado	Cambiar estado del trabajo. Body: {nuevoEstado, motivo}. Valida la transición.	ADMIN o participante
POST	/trabajos/{id}/requisitos	Añadir requisito al trabajo: descripción y URL adjunta opcional.	ADMIN
GET	/trabajos/{id}/requisitos	Listar requisitos del trabajo.	ADMIN o participante
POST	/trabajos/{id}/comentarios	Crear comentario asociado al trabajo y al usuario autenticado.	ADMIN o participante
GET	/trabajos/{id}/comentarios	Listar comentarios del trabajo, ordenados por fecha.	ADMIN o participante
GET	/trabajos/{id}/historial	Consultar historial completo de cambios de estado del trabajo.	ADMIN o participante

4.6. SEGURIDAD JWT Y MATRIZ DE PERMISOS

4.6.1. Roles y matriz de permisos

La gestión de permisos y la implementación de seguridad mediante JWT garantizan que cada usuario acceda únicamente a las funcionalidades que le corresponden según su rol. El sistema **DesignWorks** define dos perfiles principales —Administrador y Diseñador—, y cada uno dispone de acciones específicas dentro del flujo de trabajo. Además, la autenticación y autorización se realizan mediante tokens JWT, asegurando un acceso seguro desde la aplicación móvil. El sistema diferencia dos roles operativos:

- **Administrador (ADMIN)**
  Responsable de la creación, edición y supervisión global de los trabajos. Puede gestionar participantes, definir requisitos y acceder a todos los trabajos del sistema.
- **Diseñador (DISEÑADOR)**
  Participa en los trabajos que le han sido asignados. Puede añadir comentarios, consultar detalles, cambiar estados permitidos y ver únicamente sus trabajos asignados.

La siguiente matriz resume de forma clara las acciones permitidas por cada rol:

Acción	ADMIN	DISEÑADOR
Crear trabajo	✓ Sí	✕ No
Editar trabajo	✓ Sí	✕ No
Cancelar trabajo	✓ Sí	✕ No
Ver todos los trabajos	✓ Sí	✕ No
Ver trabajos asignados	✓ Sí	✓ Sí
Añadir comentario	✓ Sí	✓ Sí (si participa)
Cambiar estado	✓ Sí	✓ Sí (si participa)
Ver historial	✓ Sí	✓ Sí (si participa)
Definir requisitos	✓ Sí	✕ No

Esta estructura permite un control estricto de las operaciones sensibles del sistema, evitando que los diseñadores puedan modificar o crear trabajos sin autorización, pero manteniendo su capacidad de colaborar dentro de los proyectos asignados.

4.6.2. Flujo de autenticación y autorización

El sistema utiliza JSON Web Tokens (JWT) como mecanismo de autenticación. Este enfoque permite un acceso seguro desde la app móvil sin necesidad de mantener sesiones en el servidor y garantiza que cada petición incluya la información necesaria para validar permisos. El flujo de seguridad funciona de la siguiente manera:

- El **usuario envía su email y contraseña** desde la aplicación.
- La **API valida las credenciales** consultando la base de datos.
- Si son correctas, la API **genera un token JWT** que contiene:
  - identificador del usuario.
  - Email.
  - rol (ADMIN o DISEÑADOR).
  - fecha de expiración.
- La App guarda el token de forma segura en “flutter_secure_storage”.
- En cada petición posterior, la App envía el token en la cabecera:
  - Authorization: Bearer {token}.
- Un **filtro de Spring Security** intercepta la petición, valida la firma, el contenido y la expiración del token.
- Finalmente, el rol incluido en el **JWT determina el acceso** a cada endpoint, aplicando la matriz de permisos definida.

Este mecanismo asegura que solo usuarios autenticados y con permisos adecuados puedan ejecutar operaciones sensibles, manteniendo así la integridad del sistema y la protección de los datos.

4.7. DISEÑO UI/UX

El diseño de la interfaz se elaboró íntegramente en Figma antes de la implementación, siguiendo los principios de Material Design 3. El prototipo de alta fidelidad incluye sistema de colores y tipografías coherente, flujos de interacción definidos y todas las pantallas principales diseñadas.

Pantalla	Funcionalidad principal
Login	Autenticación mediante email y contraseña. Validación de campos, mensajes de error y acceso al flujo principal.
Home	Dashboard con estadísticas de trabajos por estado y saludo personalizado al usuario.
Lista de trabajos (ADMIN)	Todos los trabajos del sistema con filtros por estado, prioridad, cliente y fechas. Posibilidad de crear un trabajo desde la app.
Mis trabajos (DISENADOR)	Trabajos asignados al diseñador autenticado, filtrable por estado.
Detalle del trabajo	Toda la información del trabajo: datos, requisitos, participantes, estado, historial y comentarios. Acciones: cambiar estado y añadir comentario.
Nuevo trabajo	Formulario de creación con selector de prioridad, fecha límite y diálogo de confirmación al salir.
Perfil	Datos del usuario, rol, estadísticas personales y botón de cierre de sesión.

5. IMPLEMENTACIÓN

5.1. TECNOLOGÍAS UTILIZADAS

Backend

Tecnología	Versión	Uso
Java	17	Lenguaje principal del backend.
Spring Boot	3.x	Framework base para la API REST.
Spring Security	6.x	Control de acceso y autenticación.
Spring Data JPA	3.x	Capa de acceso a datos con Hibernate.
JWT (jjwt)	0.11+	Generación y validación de tokens.
MariaDB	11	Base de datos relacional.
Docker Compose	—	Orquestación de contenedores.
Lombok	—	Reducción de código boilerplate.
Maven	—	Gestión de dependencias y build.

Frontend

Tecnología	Versión	Uso
Flutter	3.7+	Framework multiplataforma para la app móvil.
Dart	3.7+	Lenguaje de programación del frontend.
Riverpod	2.5+	Gestión de estado global reactiva.
GoRouter	14.2+	Navegación declarativa y guards de autenticación.
Dio	5.4+	Cliente HTTP con interceptores para inyección automática de tokens.
flutter_secure_storage	9.2+	Almacenamiento seguro del token JWT.
json_serializable	6.8+	Serialización/deserialización JSON de modelos.
intl	—	Formateo de fechas y localización.

5.2. FUNCIONALIDADES IMPLEMENTADAS

Módulo	Componentes	Estado
Autenticación	AuthController, AuthService, JwtService, JwtAuthFilter, SecurityConfig	Completo
Trabajos	TrabajoController, TrabajoService, TrabajoRepository + DTOs completos	Completo
Comentarios	ComentarioController, ComentarioService, ComentarioRepository	Completo
Requisitos	RequisitoController, RequisitoService, RequisitoRepository	Completo
Historial	HistorialController, HistorialService, HistorialEstadoRepository	Completo
Flutter — Auth	LoginScreen, AuthProvider, AuthService, SecureStorageService	Completo
Flutter — Trabajos	TrabajosListScreen, MisTrabajosScreen, TrabajoDetailScreen, TrabajoNuevoScreen	Completo
Flutter — Home/Perfil	HomeScreen con dashboard, PerfilScreen con estadísticas y logout	Completo
Tests automatizados	Tests unitarios backend y tests de widgets Flutter	Pendiente

5.3. INFRAESTRUCTURA Y DESPLIEGUE

Servicio	Puerto macOS	Puerto Windows	Descripción
Spring Boot API	8080	8082	API REST del backend.
MariaDB	3306	3306	Base de datos relacional.
Adminer	8081	8081	Interfaz web de administración de la BD.

Variables de entorno del backend**: DB_URL, DB_USER, DB_PASS**.

Variable del frontend: **API_BASE_URL (http://10.0.2.2:8080 en emulador, IP local en dispositivo físico)**.

6. RESULTADOS Y ESTADO DEL PROYECTO

6.1. PROGRESO GLOBAL

Componente	Progreso	Estado
Análisis	100%	Completado
Diseño	100%	Completado
Infraestructura (Docker)	95%	Funcional
Backend (Spring Boot)	95%	Funcional
Frontend (Flutter)	100%	Funcional
Diseño UI/UX (Figma + implementación)	100%	Completado
Tests manuales documentados	100%	Completado

6.2. MÉTRICAS DE CÓDIGO

Métrica	Backend	Frontend
Líneas de código aproximadas	~4.500	~3.200
Clases / Widgets	45+ clases Java	25+ widgets, 8 pantallas
Endpoints funcionales	15	—
Tablas de BD / Modelos de datos	6 tablas	10+ modelos con serialización JSON

6.3. DECISIONES TÉCNICAS RELEVANTES

- **Riverpod sobre Provider:** mejor manejo de estado reactivo y mayor modernidad.
- **GoRouter sobre Navigator:** navegación declarativa y type-safe con guards de autenticación.
- **Dio sobre http:** interceptores para inyección automática de token y mejor manejo de errores.
- **JWT sin refresh token inicialmente:** por simplicidad; se añadirá como mejora futura.
- **Diseño en Figma antes de codificar:** asegurar coherencia visual antes de implementar.
- **Arquitectura por capas en el backend:** favorece la mantenibilidad y la separación de responsabilidades.

7. CONCLUSIONES

7.1. VALORACIÓN DEL PROYECTO

El proyecto **DesignWorks** ha cumplido los objetivos principales establecidos en la fase de análisis. El sistema es funcional en su totalidad, con un backend robusto que implementa autenticación JWT, control de acceso por roles, validación de transiciones de estado y trazabilidad completa mediante historial. El frontend ofrece una interfaz intuitiva y coherente con el diseño elaborado en Figma.

La separación en capas del backend ha facilitado el mantenimiento y la legibilidad del código. El uso de Riverpod en Flutter ha permitido una gestión del estado reactiva y predecible. La documentación exhaustiva previa (fases de análisis y diseño) fue determinante para agilizar el desarrollo y evitar decisiones tardías de arquitectura.

7.2. LECCIONES APRENDIDAS

- La documentación previa (fases de análisis y diseño) fue fundamental para agilizar el desarrollo y evitar decisiones de diseño tardías.
- Diseñar en Figma antes de codificar la interfaz ahorró tiempo y mejoró la coherencia visual del resultado final.
- Docker Compose simplificó enormemente la gestión del entorno de desarrollo en múltiples equipos.
- Riverpod presenta una curva de aprendizaje, pero su sistema de providers reactivos resulta muy potente para aplicaciones complejas.
- La implementación de JWT requiere atención especial a la seguridad de los endpoints, la gestión del token en el cliente y su renovación.

7.3. TRABAJO FUTURO

- Implementar refresh token para mejorar la experiencia de usuario sin comprometer la seguridad.
- Añadir tests unitarios automatizados en el backend con cobertura mínima del 60%.
- Implementar tests de widgets en Flutter.
- Desarrollar el módulo de notificaciones push para avisos de cambios de estado.
- Añadir soporte para adjuntos de archivos reales en requisitos.
- Implementar modo oscuro en la interfaz Flutter.
- Preparar el despliegue en producción con pipeline CI/CD.

8. BIBLIOGRAFÍA Y WEBGRAFÍA

8.1. DOCUMENTACIÓN OFICIAL

- Spring Framework. (2024). Spring Boot Reference Documentation. Recuperado de https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Security. (2024). Spring Security Reference Documentation. Recuperado de https://docs.spring.io/spring-security/reference/index.html
- Flutter. (2024). Flutter Documentation — Build apps for any screen. Recuperado de https://docs.flutter.dev
- Dart. (2024). Dart Programming Language — Documentation. Recuperado de https://dart.dev/guides
- Riverpod. (2024). Riverpod — A simple way to access state. Recuperado de https://riverpod.dev
- GoRouter. (2024). GoRouter for Flutter. Recuperado de https://pub.dev/packages/go_router
- MariaDB Foundation. (2024). MariaDB Server Documentation. Recuperado de https://mariadb.com/kb/en/documentation/
- Docker Inc. (2024). Docker Compose Overview. Recuperado de https://docs.docker.com/compose/

8.2. LIBROS Y ARTÍCULOS DE REFERENCIA

- Walls, C. (2022). Spring in Action, 6th Edition. Manning Publications.
- Burd, B. (2020). Flutter for Dummies. Wiley.
- Jones, M. B., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT) — RFC 7519. IETF. Recuperado de https://datatracker.ietf.org/doc/html/rfc7519
- OWASP Foundation. (2024). REST Security Cheat Sheet. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

8.3. RECURSOS COMPLEMENTARIOS

- Baeldung. (2024). Spring Boot & Spring Security Tutorials. Recuperado de https://www.baeldung.com
- Dart pub.dev. (2024). Repositorio oficial de paquetes Dart y Flutter. Recuperado de https://pub.dev
- Figma Inc. (2024). Figma Design Tool. Recuperado de https://www.figma.com
- Material Design. (2024). Material Design 3 Guidelines. Recuperado de https://m3.material.io
- Dio Package. (2024). A powerful HTTP client for Dart/Flutter. Recuperado de https://pub.dev/packages/dio
- JetBrains. (2024). IntelliJ IDEA — IDE para desarrollo Java/Spring. Recuperado de https://www.jetbrains.com/idea/